Запрос API: Егор (Storage + Context Engine)

Егор, оркестратору от тебя нужны два объекта: **Storage** (транзакции + 7 репозиториев) и **Context Engine** (один метод `gather`). Ниже — что именно я вызываю, в какой момент и какие гарантии мне нужны. Схему таблиц, драйвер и алгоритм поиска контекста выбираешь сам. Модели данных (`Task`, `Message`, `Question` и т.д.) описаны в разделе 2 вкладки «Оркестрация».

1. Когда оркестратор к тебе обращается

Каждая строка — одна транзакция (`async with storage.unit_of_work() as uow`). Внутри транзакций я не вызываю LLM и инструменты, поэтому они короткие: 1–6 запросов.

Момент в оркестрации	Вызовы в одной транзакции
Пришло сообщение пользователя	`inbox.get(client_message_id)`, `tasks.find_active_by_chat(chat_id)`
Создание задачи	`tasks.create`, `messages.add(request)`, `inbox.put`, `snapshots.add`
Любая смена статуса	`tasks.update` (CAS), `snapshots.add`
Перед проверкой данных (`assess`)	`messages.list_by_task`, `clarifications.list_questions`, `clarifications.list_answers`, `projects.list_by_chat`, `projects.get`
Задаём вопросы	`clarifications.add_questions`, `messages.add(question)`, `tasks.update`, `snapshots.add`
Получен ответ	`messages.add(answer)`, `inbox.put`, `clarifications.add_answers`, `clarifications.set_status` × 2, `tasks.update`, `snapshots.add`
Начало шага выполнения	`messages.list_by_task`, `projects.get`, `tool_runs.list_by_task`
Перед вызовом инструмента	`messages.add(tool_call)`, `tool_runs.add(status=started)`

Момент в оркестрации	Вызовы в одной транзакции
После вызова инструмента	`tool_runs.update`, `messages.add(tool_result)`, `tasks.update`, `snapshots.add`
Завершение, ошибка, отмена	`messages.add(final / error)`, `clarifications.list_questions` + `set_status(skipped)`, `tasks.update`, `snapshots.add`
Возобновление после сбоя	`messages.list_by_task`, `tool_runs.list_by_task`, `messages.add`, `tool_runs.update`

Context Engine я вызываю вне транзакций:

- при каждом входе в `analyzing` — перед `assess`, со scopes `chat`, `project`, `memory` и бюджетом 8 000 токенов;
- один раз за запуск в `executing` (дальше результат кешируется);
- из инструментов `get_project_context` (scope `project`) и `search_memory` (scopes `memory`, `other_chats`) с бюджетом 2 000 токенов.

2. Интерфейс

```
class Storage(Protocol):
    def unit_of_work(self) -> AbstractAsyncContextManager[UnitOfWork]: ...
    # нормальный выход из блока = COMMIT; исключение = ROLLBACK + исключение
летит дальше

class UnitOfWork(Protocol):
    tasks: TaskRepo
    messages: MessageRepo
    clarifications: ClarificationRepo
    tool_runs: ToolRunRepo
    snapshots: SnapshotRepo
    projects: ProjectRepo
    inbox: InboxRepo

class TaskRepo(Protocol):
    async def create(self, task: Task) -> None
    async def get(self, task_id: str) -> Task | None
    async def find_active_by_chat(self, chat_id: str) -> Task | None
    async def update(self, task: Task) -> Task            # CAS по version,
см. гарантии
```

```
class MessageRepo(Protocol):
    async def add(self, msg: Message) -> None
    async def list_by_task(self, task_id: str) -> list[Message]
    async def list_recent_by_chat(self, chat_id: str, limit: int) ->
list[Message]

class ClarificationRepo(Protocol):
    async def add_questions(self, questions: list[Question]) -> None
    async def list_questions(self, task_id: str) -> list[Question]
    async def set_status(self, question_ids: list[str], status:
QuestionStatus) -> None
    async def add_answers(self, answers: list[Answer]) -> None
    async def list_answers(self, task_id: str) -> list[Answer]

class ToolRunRepo(Protocol):
    async def add(self, run: ToolRun) -> None
    async def update(self, run: ToolRun) -> None
    async def list_by_task(self, task_id: str) -> list[ToolRun]

class SnapshotRepo(Protocol):
    async def add(self, snapshot: TaskSnapshot) -> None
    async def list_by_task(self, task_id: str) -> list[TaskSnapshot]

class ProjectRepo(Protocol):
    async def get(self, project_id: str) -> Project | None      # Project(id,
name, root_path)
    async def list_by_chat(self, chat_id: str) -> list[Project]

class InboxRepo(Protocol):
    async def get(self, key: str) -> str | None                  #
client_message_id -> task_id
    async def put(self, key: str, task_id: str) -> None

class ContextEngine(Protocol):
    async def gather(self, query: ContextQuery) -> ContextBundle

class ContextQuery:  task_id; user_id; chat_id; project_id | None; text;
scopes: list[chat|project|other_chats|memory]; token_budget
class ContextBundle: blocks: list[ContextBlock]; total_tokens; truncated
class ContextBlock:  id; sha256; source(kind, ref); text; summary | None;
score; tokens
```

3. Гарантии, на которые опирается оркестратор

1. **Атомарность.** Всё внутри одного `unit_of_work()` фиксируется целиком или не фиксируется вообще. Иначе после сбоя будут, например, вопросы без статуса `waiting_user`.
2. **CAS в `tasks.update`.** Запись только если в БД `version == task.version` (`UPDATE … WHERE id = :id AND version = :version`). Возвращается новая копия с `version + 1` и новым `updated_at`, переданный объект не меняется. Если обновлено 0 строк — `ConcurrencyError`. На этом держится отмена задачи во время работы.
3. **Одна активная задача на чат.** `tasks.create` бросает `ConcurrencyError`, если в чате уже есть задача не в `completed / failed / cancelled`. В PostgreSQL — частичный уникальный индекс.
4. **Порядок.** Все `list_*` возвращают записи в порядке добавления. `list_recent_by_chat` — последние `limit` сообщений, но тоже в хронологическом порядке. Для переписки с LLM это критично: результат инструмента обязан идти после его вызова.
5. **Без потерь.** Модель, записанная и прочитанная обратно, равна исходной (включая `frame`, `counters`, `tool_calls`). Запись через `model_dump(mode="json")` в JSONB, чтение через `Model.model_validate`.
6. **Копии, а не ссылки.** Если будешь кешировать в памяти — отдавай копии: я меняю прочитанные объекты локально до записи.
7. **Id и время.** Id генерирует вызывающий (`task_<hex12>`), БД их не меняет. Время — UTC с часовым поясом (`timestamptz`).
8. **Ошибки.** Всё, кроме `ConcurrencyError`, я пробрасываю наружу без изменения статуса задачи — потом её можно продолжить через `resume`.

4. Требования к Context Engine

- Источники берутся только из `query.scopes`. Права доступа проверяются внутри движка по `user_id` и `project_id`; оркестратор их не фильтрует.
- `sum(block.tokens) <= token_budget`. Если что-то не влезло — `truncated = True`.
- Блоки отсортированы по `score` по убыванию, `sha256 = SHA-256(text)`, дубли с одинаковым хешем убраны.
- `source.ref` позволяет найти оригинал: `message:<id>`, `file:<путь>`, `memory:<id>`. По `ContextBlock.id` я должен потом объяснить пользователю, откуда взят контекст.

- Сообщения текущей задачи (`task_id == query.task_id`) в контекст не включать: они и так идут в промпт.
- Ничего не нашлось — вернуть пустой `ContextBundle`, а не исключение. Любое исключение из `gather` переводит задачу в `failed(internal_error)`.
- `gather` не меняет задачу. Сохранять блоки в `context_blocks` (с дедупликацией по хешу в пределах области доступа) — твоё решение. Id блоков я записываю в `task.context_block_ids`.
- Желательно укладываться в 1 с — вызов стоит на пути каждого ответа.

5. Предлагаемая схема таблиц

Это предложение, а не требование. Обязательны только гарантии из раздела 3.

```
tasks(id text pk, chat_id, user_id, project_id, request_text, status,
      frame jsonb, counters jsonb, failure jsonb, context_block_ids text[],
      version int, created_at timestamptz, updated_at timestamptz)
CREATE UNIQUE INDEX one_active_task_per_chat ON tasks(chat_id)
  WHERE status NOT IN ('completed', 'failed', 'cancelled');

messages(seq bigserial pk, id text unique, chat_id, task_id, role, kind,
         content text, tool_calls jsonb, tool_call_id, created_at)
clarification_questions(id pk, task_id, gap_id, slot_name, text, options
jsonb,
                        priority real, round int, status, created_at)
clarification_answers(id pk, question_id, task_id, value jsonb,
raw_message_id)
tool_runs(id pk, task_id, step int, call jsonb, status, result jsonb,
started_at, finished_at)
task_snapshots(id pk, task_id, version int, status, frame jsonb, counters
jsonb,
               transition jsonb, created_at)
projects(id pk, name, root_path)          chat_projects(chat_id, project_id)
inbox(key text pk, task_id)
context_blocks(id pk, sha256, scope, source_ref, text, summary, tokens)   --
по твоему усмотрению
```

6. Что решаешь сам

Схема таблиц и индексы, `asyncpg` или SQLAlchemy async, миграции, где лежат файлы проектов, алгоритм поиска и ранжирования контекста, summaries, позже pgvector.

7. Вопросы к тебе

- [ ] Ок ли тебе асинхронный интерфейс (`async def`)? Оркестратор асинхронный из-за долгих вызовов LLM.
- [ ] Как проекты привязываются к чату и кто заполняет `root_path`? Он нужен файловым инструментам.
- [ ] Что будет лежать в scope `memory` и как решается доступ к `other_chats`?
- [ ] Нужны ли тебе от оркестратора ещё какие-то данные в `ContextQuery`?
- [ ] Когда реально сделать первую версию, хотя бы в памяти, чтобы начать интеграцию?